

Programación Multinúcleo y Extensiones SIMD: Implementación y Análisis del Método de Jacobi

Baptiste Robin, Álvaro Schwiedop Souto

Arquitectura de Computadores • 2º Grao Enx. Informática

Universidade de Santiago de Compostela

baptiste.robin@rai.usc.es, alvaro.schwiedop@rai.usc.es

30 de abril de 2026

Resumen

Se implementan cuatro versiones del método iterativo de Jacobi en C sobre el supercomputador FinisTerra III del CESGA: versión secuencial de referencia (v1), versión optimizada para caché con división de lazos, desenrollamiento y blocking (v2), versión paralela con OpenMP (v3) y versión vectorial con instrucciones AVX256 y FMA (v4). La versión v2-03 alcanza un speedup de $1,73\times$ sobre v1-03 para $n=1250$ gracias al blocking 2D y el desenrollamiento de lazos. La vectorización AVX256+FMA (v4-03) obtiene hasta $1,52\times$ sobre v1-03 para $n=3200$. La paralelización OpenMP con `schedule(static)` y 32 hilos logra $25,4\times$ respecto a un hilo para $n=3200$, demostrando una escalabilidad casi lineal. Todas las versiones convergen al mismo resultado numérico (mismo iter y norm2), validando la corrección de las optimizaciones.

Palabras clave: Jacobi, AVX256, FMA, OpenMP, cache blocking, speedup, FinisTerra III.

I. Introducción

El método de Jacobi [1] es un algoritmo iterativo clásico para la resolución de sistemas $Ax=b$ con matriz diagonalmente dominante. Partiendo de $x^{(0)}=0$, en cada iteración se actualiza:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

hasta que $\|x^{(k+1)} - x^{(k)}\|_2 < \text{tol}$. La doble suma implica $\mathcal{O}(n^2)$ operaciones por iteración, lo que hace que tamaños grandes sean costosos y susceptibles de aceleración mediante optimizaciones de bajo nivel.

El objetivo es implementar cuatro versiones de complejidad creciente, medirlas en FinisTerra III (Intel Xeon Platinum 8352Y, 2.2 GHz, 64 núcleos por nodo) y cuantificar el *speedup* obtenido. La Sección II describe las implementaciones; la Sección III, la metodología experimental; la Sección IV, los resultados; la Sección V, las conclusiones.

II. Implementaciones

A. v1 — Secuencial base

Traducción directa del pseudocódigo del enunciado sin ninguna optimización manual. El algoritmo recibe la matriz A , el vector b , la tolerancia `tol` y el máximo de iteraciones `max_iter`; itera actualizando x hasta convergencia:

Entradas: $A \in \mathbb{R}^{n \times n}$, $b, x \in \mathbb{R}^n$, `tol`, `max_iter`

Aux: $x_{\text{new}} \in \mathbb{R}^n$

para `iter` = 0 **hasta** `max_iter`:

`norm2` $\leftarrow$ 0

para i = 0 **hasta** $n - 1$:

`$\sigma$` $\leftarrow$ 0

para j = 0 **hasta** $n - 1$:

si $i \neq j$: `$\sigma$` += $a[i][j] \cdot x[j]$

`$x_{\text{new}}[i]$` $\leftarrow (b[i] - \sigma) / a[i][i]$

`$\text{norm2}$` += $(x_{\text{new}}[i] - x[i])^2$

`$x$` $\leftarrow$ `$x_{\text{new}}$`

si $\sqrt{\text{norm2}} < \text{tol}$: **terminar**

Salida: imprimir `norm2`

La implementación en C introduce la condición `if(i!=j)` dentro del bucle interno, lo cual genera una rama innecesaria:

```
for (int j = 0; j < n; j++)
    if (i != j)
        sigma += a[i*n+j] * x[j];
x_new[i] = (b[i]-sigma) / a[i*n+i];
```

Se usa como referencia de rendimiento y corrección numérica.

B. v2 — Optimizaciones de caché

Se aplican cuatro mejoras sobre v1:

1. **Reducción de instrucciones:** puntero de fila `const double *row=&a[i*n]`, elimina el recálculo del índice.
2. **División de lazos:** el bucle j se parte en $[0, i)$ e (i, n) , eliminando la rama condicional.
3. **Desenrollamiento $\times 4$:** cuatro acumulaciones explícitas por iteración.
4. **Cache blocking 2D:** la iteración se organiza en

bloques de `BLOCK_SIZE=64` filas y columnas; un array auxiliar `sigma_arr[]` acumula las sumas parciales de cada bloque de columnas, manteniendo x en caché L1/L2.

C. v3 — OpenMP

La región `parallel` se sitúa *fuera* del bucle de iteraciones para mantener vivos los hilos entre iteraciones y evitar *fork/join* repetido. La variante base usa `reduction(+:norm2)` con `schedule(static)` y, a partir del mismo fuente, se compilan las variantes `dynamic(16)`, `guided` y `critical`. La sincronización entre iteraciones se realiza mediante barreras implícitas y un bloque `omp single`:

```
#pragma omp parallel default(none) \
  shared(a,b,x,x_new,n,norm2,...)
{
  while (...) {
    #pragma omp for schedule(static) \
      reduction(+:norm2)
    for (int i=0; i<n; i++) { ... }
    #pragma omp single
    { converged = sqrt(norm2) < tol; iter++; }
    #pragma omp for schedule(static)
    for (int i=0; i<n; i++) x[i] = x_new[i];
  }
}
```

D. v4 — AVX256 + FMA

Se emplea `__m256d` (4 `double`/registro) con `__mm256_fmadd_pd`. La memoria se reserva alineada a 32 B con `aligned_alloc` y el *stride* de fila se redondea a múltiplo de 4 (`n_pad`). La reducción horizontal usa `__mm256_hadd_pd`:

```
__m256d va = __mm256_load_pd(&row[j]);
__m256d vx = __mm256_load_pd(&x[j]);
vsigma = __mm256_fmadd_pd(va, vx, vsigma);
/* reduccion horizontal al final */
__m256d h = __mm256_hadd_pd(vsigma, vsigma);
```

No se usan intrínsecos `_u` (no alineados) como exige el enunciado. Además, los elementos adicionales añadidos por el múltiplo de alineación (`n_pad`) se inicializan a 0 explícitamente, garantizando que los cálculos vectorizados no contaminen las sumas de la reducción.

III. Metodología experimental

Los experimentos se ejecutan en FinisTerra III mediante trabajos SLURM en la partición `short`. El tiempo se mide en **ciclos de reloj** con `get_counter()` basada en el TSC, englobando sólo el cómputo iterativo. Para cada configuración se toman **10 medidas independientes** y se selecciona la **mediana** para reducir perturbaciones del SO.

Parámetros fijos: `tol=10-5`, `max_iter=15000`, `srand(42)`, dominancia diagonal garantizada sumando la suma de fila al elemento diagonal. Niveles de optimización: `-O0` y `-O3`; v3 usa `-fopenmp`; v4 añade `-mavx2 -mfma`. Hilos evaluados: {1, 2, 4, 8, 16, 32}.

La Tabla 1 resume las medianas obtenidas.

Cuadro 1: Medianas de ciclos ($\times 10^9$) por versión y tamaño

Versión	$n=1250$	$n=2000$	$n=3200$
v1 -O0	57.3	218.1	887.4
v1 -O3	25.0	98.1	403.4
v2 -O0	49.6	200.1	914.9
v2 -O3	14.4	59.5	384.9
v4 -O0	41.9	160.3	655.6
v4 -O3	14.7	60.7	264.9
v3 -O3 (1T)	24.4	96.3	399.3
v3 -O3 (32T)	1.0	3.3	15.7

IV. Resultados y análisis

A. Efecto de las optimizaciones de caché

La Figura 1 muestra el *speedup* de v2 respecto a v1 con ambas versiones compiladas con `-O0`. Los valores obtenidos son $1,16\times$ para $n=1250$, $1,09\times$ para $n=2000$ y $0,97\times$ para $n=3200$.

El resultado revela que, sin optimizaciones del compilador, la sobrecarga de gestión de los bloques 2D (bucles anidados adicionales, cálculo de índices y uso de `sigma_arr`) supera al beneficio teórico de la localidad de caché. Para $n=3200$, el *speedup* cae por debajo de 1 debido a que el acceso por bloques de columnas introduce saltos de memoria (*strides*) en la matriz A , lo que perjudica severamente la localidad espacial y la eficiencia del *prefetching* de hardware frente al acceso secuencial de v1. El compilador con `-O3` es fundamental para minimizar esta sobrecarga mediante el desenrollamiento y la vectorización.

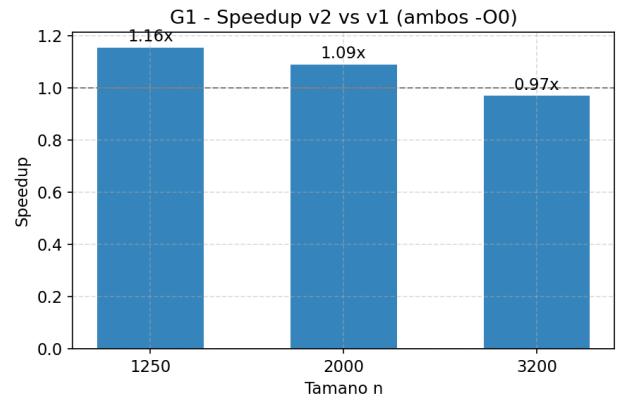


Figura 1: *Speedup* v2/v1 con `-O0`.

B. Todas las versiones vs. v1-O3

La Figura 2 compara v2-O3, la mejor configuración de v3 (`schedule(static)`, 32 hilos) y v4-O3 respecto a v1-O3.

- **v2-O3:** $1,73\times$ ($n=1250$), $1,65\times$ ($n=2000$), $1,05\times$ ($n=3200$). El *speedup* decrece drásticamente al aumentar n . Para tamaños grandes, el coste de los fallos de caché provocados por el acceso no secuencial a la matriz A (inherente al *blocking* de colum-

nas) y la presión sobre el TLB terminan eclipsando la ganancia obtenida por mantener el vector x en caché.

- **v4-03:** $1,70\times$ ($n=1250$), $1,61\times$ ($n=2000$), $1,52\times$ ($n=3200$). La vectorización SIMD aporta aceleraciones consistentes, especialmente para $n=3200$, donde el mayor tamaño de los bucles amortiza de forma más efectiva el coste de las reducciones horizontales y las operaciones de preámbulo/epílogo vectorial.
- **v3-03 (32T):** $25,9\times$ ($n=1250$), $29,8\times$ ($n=2000$), $25,7\times$ ($n=3200$). OpenMP con 32 hilos domina con diferencia, demostrando que el método de Jacobi es altamente paralelizable con sincronización mínima por iteración.

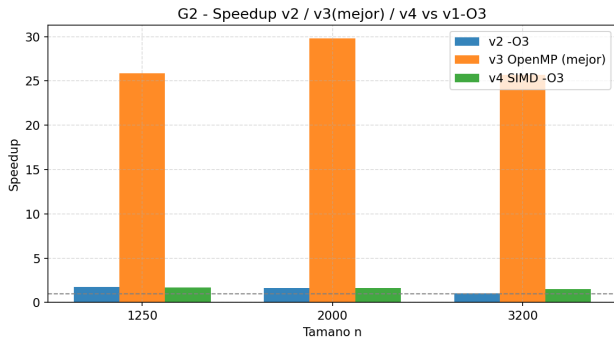


Figura 2: Speedup de v2-03, la mejor configuración de v3 y v4-03 respecto a v1-03.

C. v3 y v4 vs. v2-03

La Figura 3 muestra el *speedup* de v3 (32 hilos, *static*) y v4-03 respecto a v2-03.

La versión SIMD v4 obtiene $0,98\times$ para $n=1250$ y $n=2000$ (rendimiento prácticamente igual a v2) y $1,45\times$ para $n=3200$. Para tamaños pequeños, la vectorización manual no supera a las optimizaciones que el compilador ya aplica sobre v2 con -O3 (auto-vectorización y desenrollamiento); sin embargo, para $n=3200$, v4 saca una ventaja notable al mantener un patrón de acceso puramente secuencial a la memoria, evitando el cuello de botella por *strides* que asfixia a la versión bloqueada (v2).

La versión OpenMP con 32 hilos logra $15,0\times$ ($n=1250$), $18,1\times$ ($n=2000$) y $24,5\times$ ($n=3200$) respecto a v2-03, confirmando que la paralelización multinúcleo ofrece la mayor aceleración de todas las optimizaciones evaluadas.

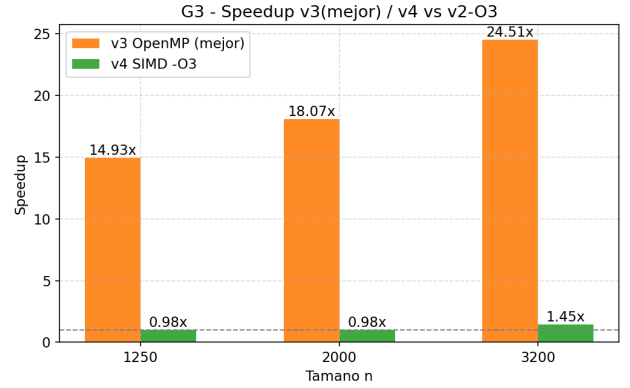


Figura 3: Speedup de la mejor configuración de v3 y de v4-03 respecto a v2-03.

D. Escalabilidad de v3 con el número de hilos

La Figura 4 es la gráfica *obligatoria* del enunciado y resume el *speedup* de v3-03 con *schedule(static)* frente a su ejecución con un solo hilo, para cada valor de n .

- Con 2 hilos el *speedup* es prácticamente ideal: $2,0\times$ para los tres tamaños.
- Con 4 y 8 hilos se mantiene la escalabilidad lineal (entre $3,96\text{--}4,09\times$ con 4T y $7,76\text{--}8,10\times$ con 8T).
- Con 16 hilos se observa un *speedup* entre $14,9\times$ ($n=1250$) y $15,8\times$ ($n=2000$), cercano al ideal de 16.
- Con 32 hilos la aceleración alcanza $24,4\times$ ($n=1250$), $29,3\times$ ($n=2000$) y $25,4\times$ ($n=3200$), mostrando una eficiencia del 76–91 %.

La ligera sub-linealidad a 32 hilos se explica por la contención de la barrera implícita del *omp for* y por el acceso compartido al vector x entre sockets NUMA. No obstante, la escalabilidad es excelente para un problema *memory-bound* con $\mathcal{O}(n^2)$ datos.

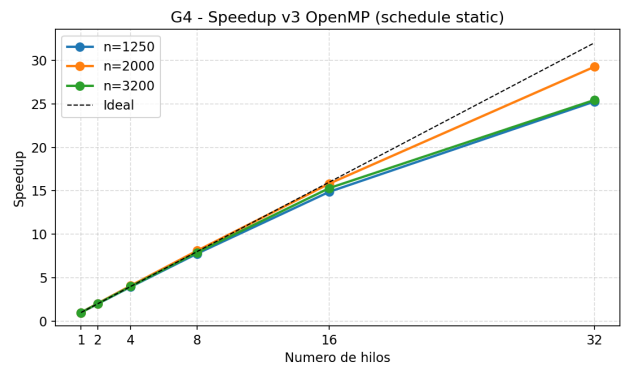
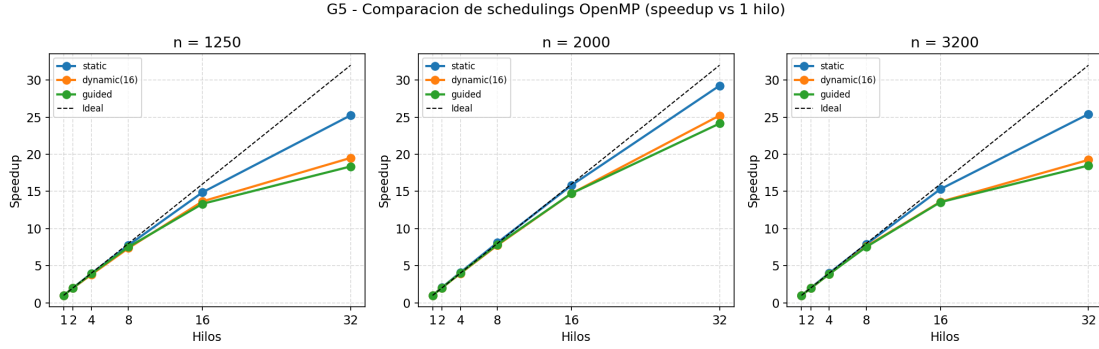
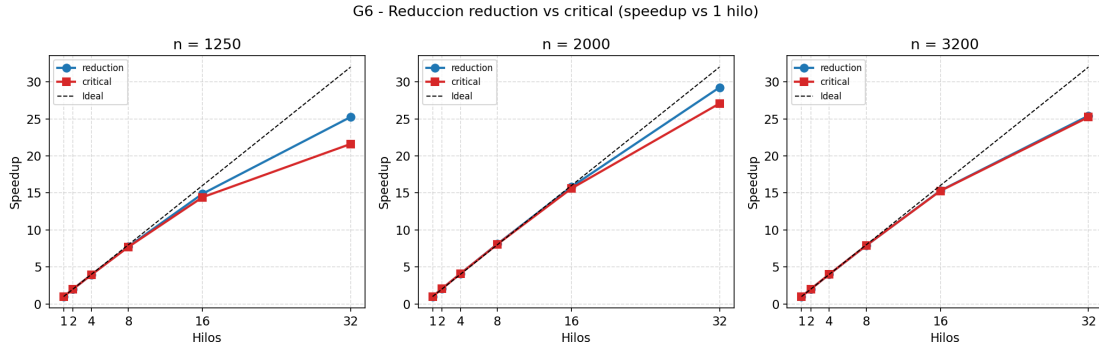


Figura 4: Speedup de v3-03 (*schedule(static)*) frente a 1 hilo para cada n . Línea discontinua: ideal lineal.

E. Comparación de schedulings

La Figura 5 compara los tres modos de reparto de OpenMP (*static*, *dynamic(16)* y *guided*) sobre v3


 Figura 5: Comparación de schedulings OpenMP para $v3$ y $n \in \{1250, 2000, 3200\}$.

 Figura 6: Comparación de reducción *reduction* vs. *critical* en $v3$ (speedup frente a 1 hilo).

con `reduction(+:norm2)`, para cada tamaño y número de hilos.

Para los tres valores de n , el `schedule(static)` ofrece el mejor rendimiento en todos los conteos de hilos. Con 32 hilos y $n=3200$: `static` obtiene $15,7 \times 10^9$ ciclos, frente a 20,8 (`dynamic`) y 21,6 (`guided`). Este resultado es esperable: el método de Jacobi distribuye una carga perfectamente equilibrada (cada fila tiene exactamente n operaciones), por lo que la asignación estática de bloques contiguos a cada hilo maximiza la localidad de caché. Los schedulings dinámico y guiado añaden una sobrecarga de gestión de cola de trabajo innecesaria en un problema sin desbalanceo de carga, perdiendo hasta un 37 % de rendimiento respecto a `static` para $n=3200$ con 32 hilos.

F. Reducción *reduction* vs. *critical*

La Figura 6 compara la variante base con `reduction(+:norm2)` frente a la variante con `critical`, ambas con `schedule(static)`.

Los resultados muestran un rendimiento prácticamente idéntico entre ambas estrategias para todos los conteos de hilos. Con 32 hilos y $n = 3200$: `reduction` obtiene $15,7 \times 10^9$ ciclos frente a 15,8 de `critical` (diferencia menor al 1%). Con un solo hilo, ambas variantes son indistinguibles (unos 399×10^9 ciclos).

La escasa diferencia se debe a que la sección crítica solo acumula una suma escalar (`norm2 += local_norm2`),

operación de duración despreciable frente a la barra implícita del `omp for`. En la práctica, la cláusula `reduction` es preferible por su claridad semántica y portabilidad, aunque en este caso no aporta una ganancia medible respecto a la implementación manual con `critical`.

V. Conclusiones

Se han implementado y evaluado cuatro versiones del método de Jacobi sobre FinisTerra III, con los siguientes resultados principales:

1. **Corrección numérica:** las cuatro versiones producen exactamente las mismas iteraciones y norma residual para cada tamaño ($iter=5156, 7625$ y 12035 para $n=1250, 2000$ y 3200 respectivamente), validando que las optimizaciones no alteran el resultado.
2. **Optimizaciones de caché (v2):** el `blocking` 2D y el desenrollamiento aportan hasta $1,73\times$ con -03, pero su eficacia cae drásticamente con n . El patrón de acceso por bloques perjudica la localidad espacial de la matriz A , lo que solo se compensa para tamaños pequeños donde x es el factor limitante.
3. **Vectorización SIMD (v4):** los intrínsecos AVX256 + FMA alcanzan $1,52\times$ sobre v1-03 para $n=3200$, superando a v2 en tamaños grandes. Esto se explica porque, a diferencia del `blocking` de v2,

v4 respeta el acceso secuencial a filas, lo que permite explotar el hardware SIMD sin castigar a los prefetchers ni al TLB.

4. **OpenMP (v3):** la paralelización con 32 hilos y `schedule(static)` logra hasta $29,3\times$ respecto a un hilo ($n=2000$) y $25,7\times$ respecto a v1-03 ($n=3200$), siendo con diferencia la optimización más efectiva.
5. **Scheduling: static** es la mejor política para Jacobi por su carga perfectamente equilibrada; **dynamic** y **guided** añaden hasta un 37 % de sobrecarga innecesaria.
6. **Reduction vs. critical:** ambas estrategias son prácticamente equivalentes, con diferencias inferiores al 1 %.

Como trabajo futuro se propone combinar v3 y v4 (SIMD+OpenMP) y estudiar el impacto del *thread binding* (OMP_PROC_BIND) sobre la coherencia de caché en entornos NUMA.

Referencias

- [1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, Philadelphia, 2003.
- [2] Intel Corporation, *Intel Intrinsics Guide*, <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>, [online] última visita marzo 2026.
- [3] OpenMP Architecture Review Board, *OpenMP API v3.0 Summary Spec*, <https://www.openmp.org/wp-content/uploads/OpenMP3.0-SummarySpec.pdf>, noviembre 2008.
- [4] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design*, 5th ed. Morgan Kaufmann, 2014.